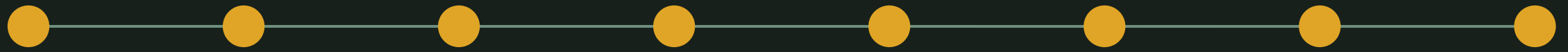
PROPOSAL · OCTOBER 2026

Bagheera

One clear way from Jira ticket to merged, reviewed, demoed code

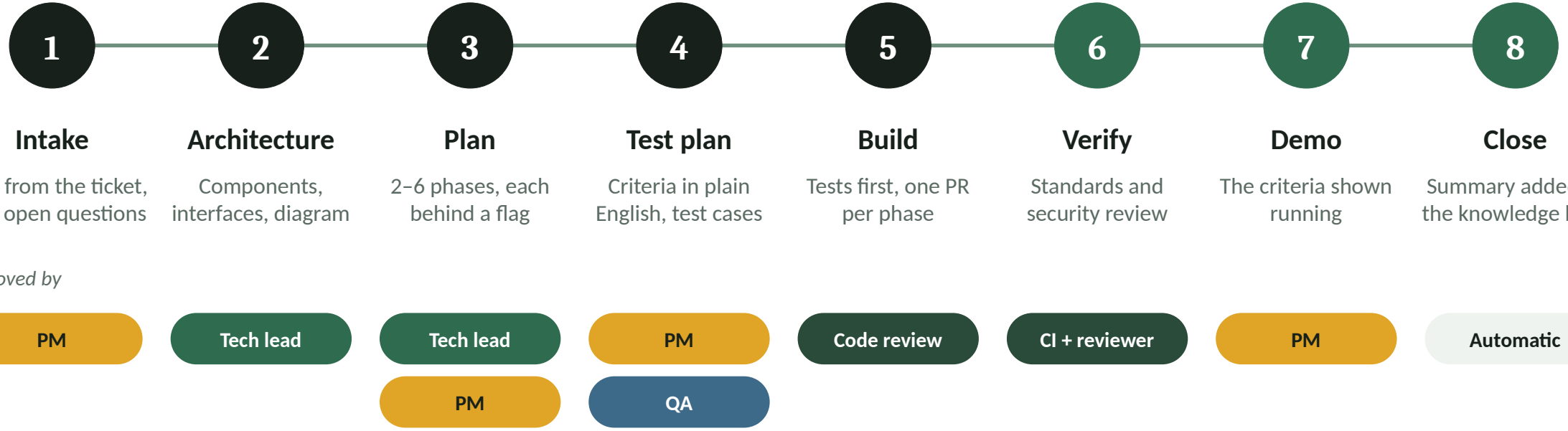
For product managers, engineering managers and the VP of Engineering



Today versus with Bagheera

	Today	With Bagheera
Knowledge	Standards, service details and past decisions live in heads, old docs and chat logs	One knowledge base that updates itself from merged work; Claude reads it in every session
Standards	Each team reads them differently; reviewers catch issues late and unevenly	Written once by their owners, checked on every PR, dangerous actions blocked
Developer path	Everyone works with AI differently; specs and plans stay in private chats	The same few commands in every repo, ticket to merged code, next step always clear
PM feedback	A ticket, then a demo weeks later; questions wait for meetings	PMs shape and approve the spec, criteria and demo in the PR, within hours
Leadership view	Status, bottlenecks and AI cost are assembled by hand	One dashboard: every story's stage, who it waits on, adoption and cost by team

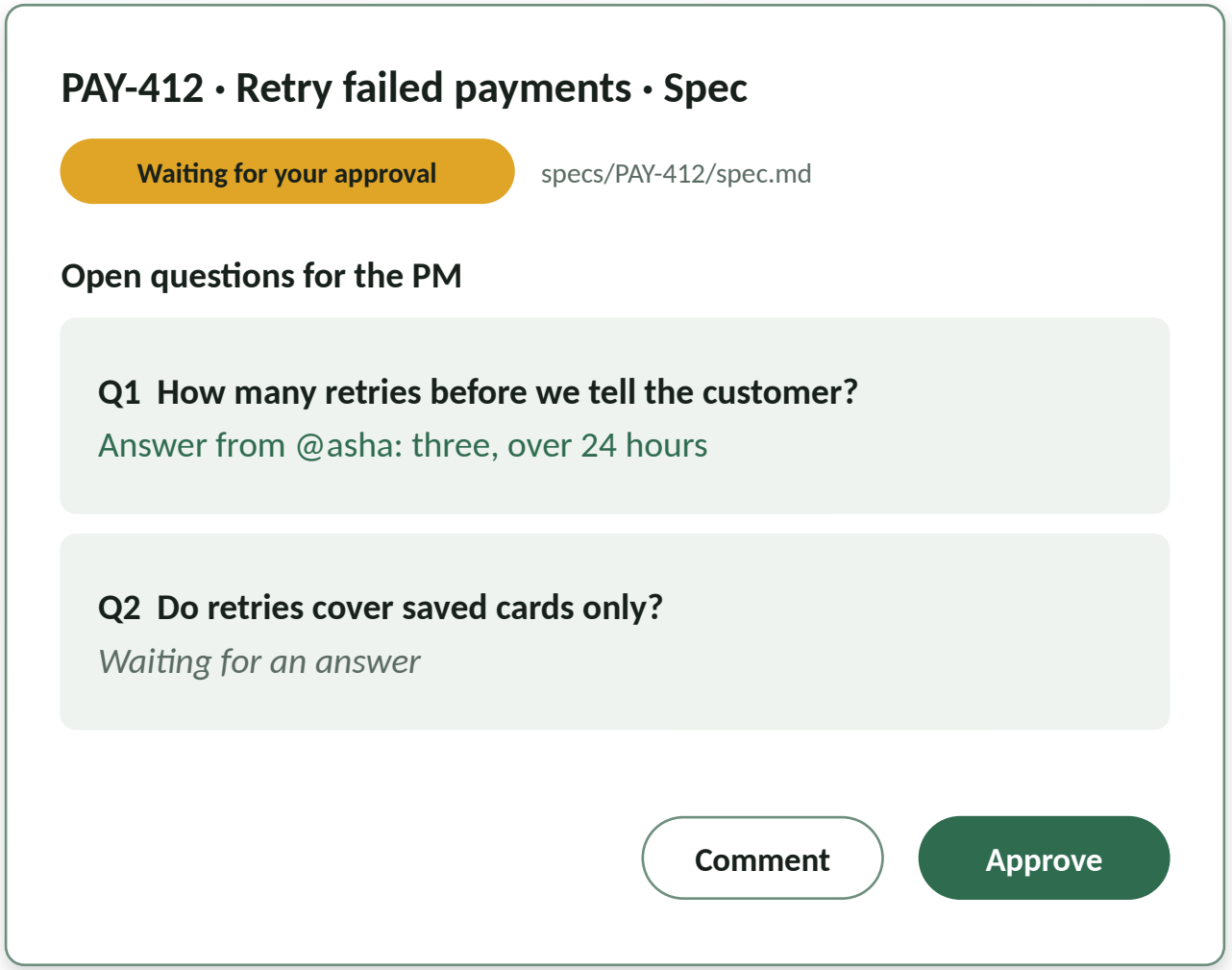
One path for every feature



The next stage waits for approval
Each stage writes a short document into the repo and opens a pull request. A local edit never counts as approval.

- Stages 1–5: one command each, on the engineer's laptop
- Stages 6–8: automatic, in CI

PMs approve in the PR, not in a meeting



Read it as a page

Specs, plans and test plans show as readable pages in GitHub. No developer tools.

Answer questions inline

Open questions are numbered; your answers flow straight into the next stage.

Know what needs you

You're notified when a review is yours, and one list shows everything waiting on you.

See which criteria pass

Each acceptance criterion gets an ID that becomes the name of its automated test.

Every story's status, without asking anyone

Waiting on me | **All** | By team | Stuck > 2 days | Shipped | Team: all | PM: all | Stage: all | Status: all

Story (opens the PR to approve)	Repo	Stage	Status	Waiting on	For	Phases
PAY-412 Retry failed payments · Test plan	payments-api	4 Test plan	Waiting for approval	PM @asha, QA @lin	2 days	0 / 3
PAY-398 Refund webhooks · Phase 2	payments-api	5 Build	In code review	@tomas (reviewer)	5 hours	1 / 4
SRCH-77 Typo-tolerant search · Spec	search-svc	1 Intake	Changes requested	@kim (developer)	1 day	–
PAY-350 Saved cards · Demo	payments-web	7 Demo	Waiting for approval	PM @asha	4 days	3 / 3

Nobody types a status

Stage, status and who it's waiting on come from the repos and pull requests.

One click to act

Each story opens the pull request that is waiting for approval.

Adoption and cost too

By team, from Claude's built-in analytics, plus a weekly one-page summary.

Standards written once, applied everywhere



Written once

- One knowledge base: standards, service pages, decisions and shipped features
- Each standard has a named owner who approves every change



Applied everywhere

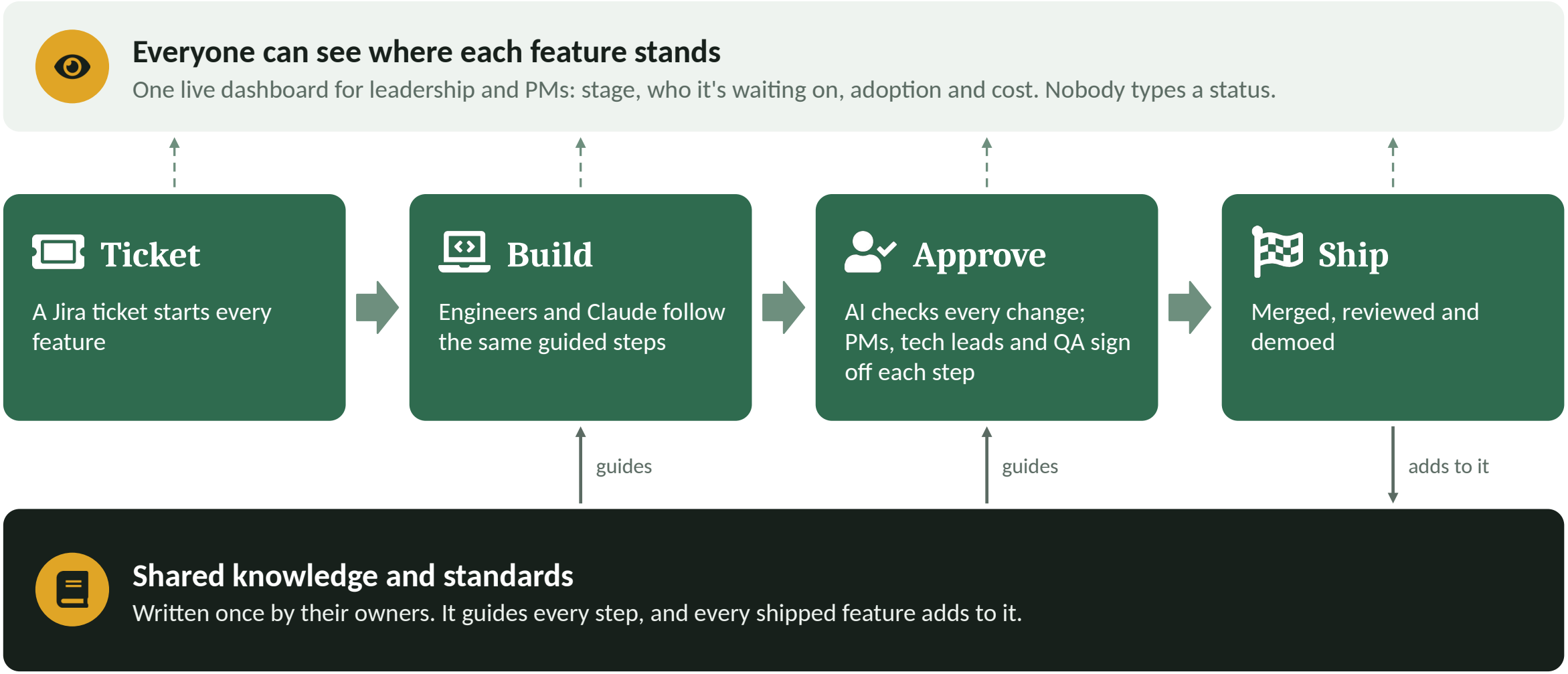
- Loaded into every Claude Code session, in every repo
- Every pull request checked against them before human review
- Deploys, force-push and production credentials blocked outright



Kept current

- Each merged change proposes updates to the pages it affects
- A weekly check flags stale, broken or contradicting pages
- Existing services start from an automatically drafted page

How it works, in one picture



Built from tools we already use

2

repos: Bagheera itself and the knowledge base

1

small script is the only new code

3

automated workflows in CI

0

new servers to run

What we configure, not build

Claude Code	One plugin for every engineer, installed and updated centrally
GitHub Spec Kit	The stage commands, with our own templates
GitHub	Pull requests, approval rules, CI and the dashboard
Jira	Tickets in, links back to the work
Claude analytics	Adoption and cost by team

Our first attempt had too many moving parts. This one is sized for a platform team of two.

Pilot first, then grow team by team

1

Foundations

Knowledge base, standards from their owners, first service pages, cost visible

2

Pilot: stages 1–4

One team, one real feature; PMs approve in PRs; dashboard live

3

Pilot: stages 5–8

Build, automated review, demo and close

4

Org rollout

2–3 teams at a time, each adding its service pages

About 6–8 weeks to the pilot · 2 platform engineers · 1 pilot team of 6–8 engineers

How we'll know it works

≥ 90%
of service pages updated within a day of a change

100%
of merged pilot PRs reviewed automatically

≥ 6
pilot engineers using it every week

2
features shipped through all eight stages

< 1 day
median time to a PM's first response

Fewer
status and clarification meetings than at the start

What we need from you

1. **A pilot team and a real feature**
   and that team's count of status and clarification meetings in week 1, as our baseline

2. **GitHub access for PMs and QA**
   with review rights on their team's repos, plus view access for leadership

3. **Our Claude plan confirmed**
   the Enterprise plan reports adoption and cost by team with no extra work

4. **A CI budget per merged PR**
   agreed before the pilot starts

5. **Backing for the pilot rule**
   we expand only once the team uses the approvals instead of skipping them

Next: the platform team settles the open design decisions, then brings a phased plan for review.